hashlock.

Security Audit

Libertum “Bonding” (DeFi)

May 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

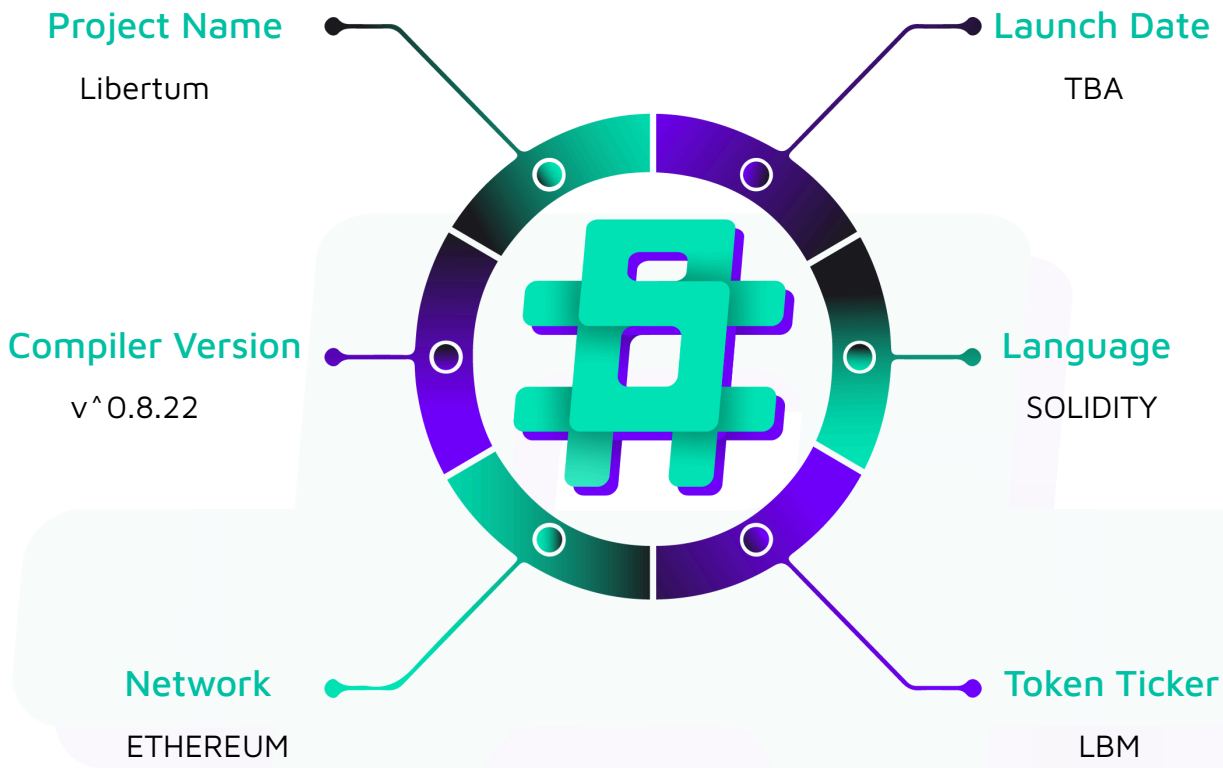
The Libertum team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

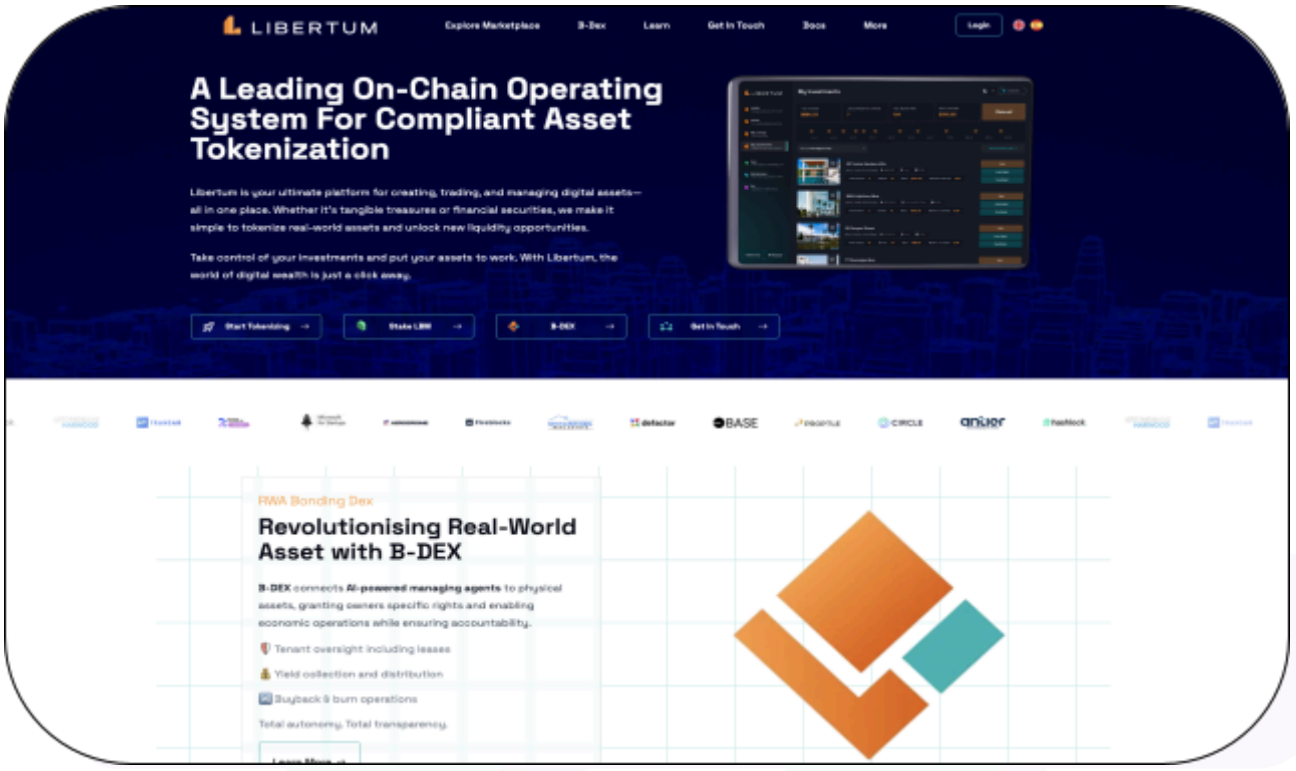
Libertum is a blockchain-based platform focused on real estate tokenization. It enables users to invest in fractional ownership of real estate assets through tokenized shares, making property investment more accessible and liquid. By leveraging DeFi principles, Libertum aims to provide passive income opportunities, transparency, and global access to real estate markets, allowing users to earn rental yields while maintaining full control over their investments via a decentralized ecosystem.

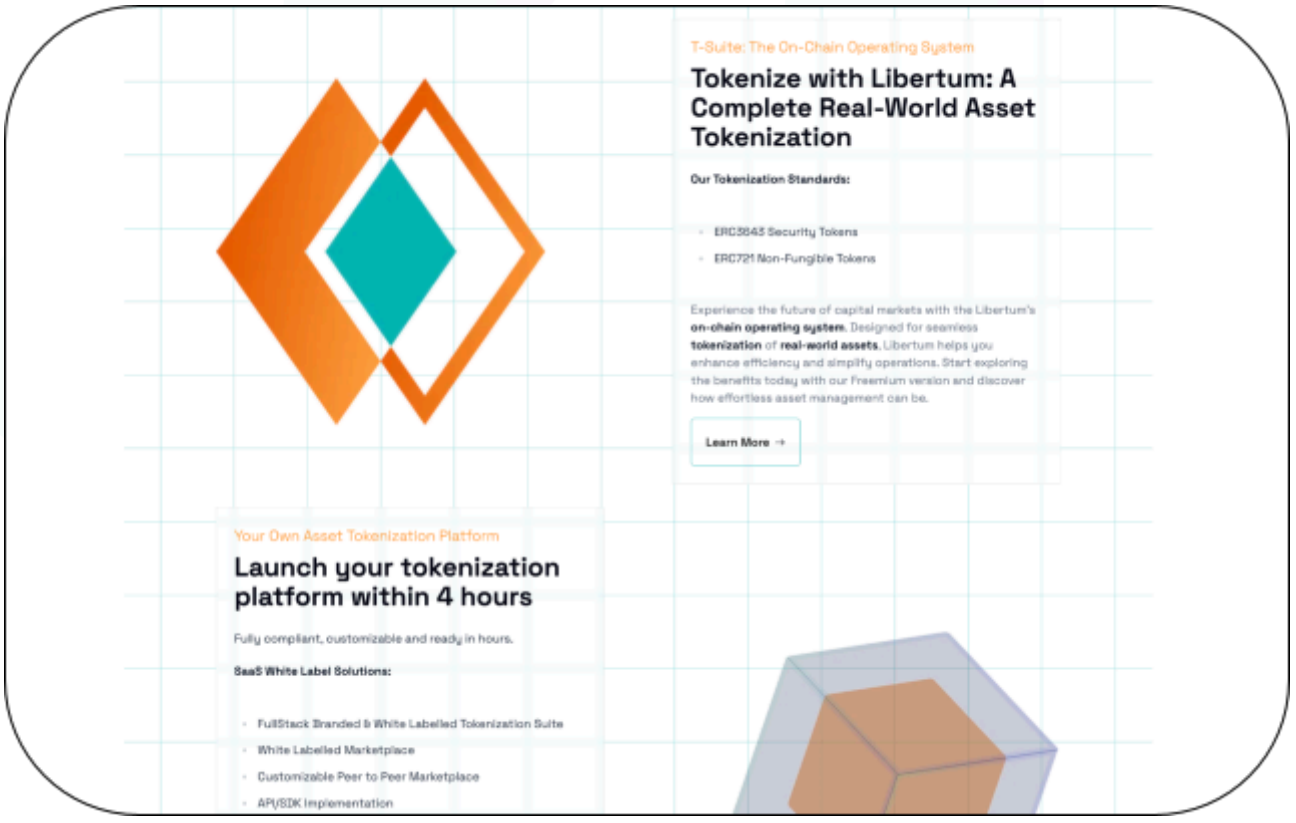
Project Name: Libertum
Project Type: DeFi
Compiler Version: ^0.8.22
Website: https://libertum.io/en/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the Libertum project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Libertum Smart Contracts
Platform	**Ethereum / Solidity**
Audit Date	**May, 2025**
Contract 1	BondingTokenController.sol
Contract 2	BondingTokenControllerProxy.sol
Contract 3	BondingTokenControllerStorage.sol
Contract 4	BondingFactory.sol
Contract 5	BondingFactoryProxy.sol
Contract 6	BondingFactoryStorage.sol
Contract 7	IBondingFactory.sol
Contract 8	BondingToken.sol
Contract 9	BondingTokenStorage.sol
Contract 10	IBondingToken.sol
Contract 11	IToken.sol
Contract 12	IImplementationAuthority.sol
Contract 13	ImplementationAuthority.sol
Contract 14	ProxyV1.sol
Audited GitHub Commit Hash	c012395bf43150204739d179bf3180ec58d85890
Fix Review GitHub Commit Hash	2349b5f5e3bf002f5c3be8218995555aaf81f322

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 High severity vulnerabilities

2 Medium severity vulnerabilities

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Controller Allows users to: - Purchase bonding tokens during the initial sale phase - Sell bonding tokens back to the bonding curve (when enabled) - Track their token purchase history and current holdings - Monitor sale progress and available token supply - View current token pricing and sale parameters Allows admins to: - Pause and unpause trading operations for emergency situations - Update platform fee percentages and parameters - Withdraw accumulated protocol fees - Modify critical system configurations	**Contract achieves this functionality.**
Factory - Deploy new bonding tokens with customizable parameters - Track all deployed tokens and map them to their creators - Maintain a registry of admin addresses with elevated permissions - Store platform-wide configuration (fee percentages, Uniswap addresses) - Calculate and enforce deployment limits per	**Contract achieves this functionality.**

creator - Manage integration points with external protocols (Uniswap V3) - Enable/disable factory operations globally	
bondingCurve - Implements ERC20 standard with additional bonding curve functionality - Manages two-phase token lifecycle: Initial Sale → DEX Trading - Executes buy operations at fixed price during sale phase - Handles sell operations with configurable restrictions - Automatically provisions liquidity to Uniswap V3 upon sale completion - Distributes funds between liquidity, platform fees, and creators - Tracks sale progress and enforces token allocation limits - Integrates with Factory for configuration and fee management	**Contract achieves this functionality.**
proxy - Routes all function calls to current implementation contracts - Maintains storage layout consistency across upgrades - Provides centralized upgrade authority management - Enables atomic upgrades across multiple contracts - Implements access control for upgrade	**Contract achieves this functionality.**

operations - Supports emergency pause functionality - Preserves state while allowing logic modifications	

Code Quality

This audit scope involves the smart contracts of the Libertum project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Libertum project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] BondingToken#buy/sell - An attacker can force any user to buy or sell tokens on their behalf

Description

The `buy()` and `sell()` functions in the BondingToken contract accept an arbitrary caller address parameter instead of using `msg.sender`. This allows any attacker to force trades on behalf of other users who have approved the contract, leading to unauthorized token purchases/sales and potential fund theft.

Vulnerability Details

The vulnerable functions accept a caller parameter that can be set to any address:

```
function buy(uint256 _amount, address caller) external WheninTrading{
    IBondingFactory bFactory = IBondingFactory(bondingFactory);
    uint256 tokenPrice = bFactory.getTokenPrice(address(this));
    address feeToken = bFactory.getFeeToken();

    // ... fee calculations ...

    // Transfers use the caller parameter, not msg.sender
    TransferHelper.safeTransferFrom(stableCoin, caller, address(this), finalCost);
    TransferHelper.safeTransferFrom(feeToken, caller, bFactory.getFeeCollector(),
finalFee);
    TransferHelper.safeTransfer(address(this), caller, _amount);

    // ...
}

function sell(uint256 _amount, address caller) external WheninTrading{
    require(_amount > 0, "Amount should be greater than 0");
```

```
    // ... price calculations ...

    // Transfers from caller, not msg.sender
    TransferHelper.safeTransferFrom(address(this), caller, address(this), _amount);
    TransferHelper.safeTransfer(stableCoin, caller, _returnAmount);
    TransferHelper.safeTransferFrom(feeToken, caller, bFactory.getFeeCollector(),
finalFee);

  // ...
}
```

This means that those users who have approved their tokens for buying and selling can be used by attackers to force buy or sell them.

Proof of Concept

Paste the following POC in your local foundry setup.

```
     function testForceBuyAndSell() public {
        uint256 buyAmount = 1e18; // 1 token

        uint256 usdcBefore = usdc.balanceOf(user1);
        uint256 tokensBefore = bondingToken.balanceOf(user1);

        // assuming User1 has already approved their tokens to the contract      for
buying and selling.
        vm.prank(user2);
        bondingToken.buy(buyAmount, user1);

        uint256 usdcAfter = usdc.balanceOf(user1);
        uint256 tokensAfter = bondingToken.balanceOf(user1);

        uint256 usdcDeducted = usdcBefore - usdcAfter;
        uint256 tokenIncrease = tokensAfter - tokensBefore;

        console.log("Balance Deducted", usdcDeducted);
```

```
        console.log("Token Increase" , tokenIncrease);

        // assuming User1 already approved their bonding tokens for selling.
        vm.prank(user1);
        bondingToken.approve(address(bondingToken), type(uint256).max);

        vm.prank(user2);
        bondingToken.sell(buyAmount, user1);

    }
```

Looking at the logs, we can clearly see how the balance of user funds was deducted by another user.

```
[PASS] testForceBuyAndSell() (gas: 238655)
Logs:
  Balance Deducted 1000000
  Token Increase 1000000000000000000
```

Impact

Users are forced to buy and sell tokens without their permission, provided that they have already approved the amount which is highly likely.

Recommendation

Fix the code in such a way that transfer of stablecoin is happening from msg.sender instead of the caller.

Status

Resolved

[H-02] BondingToken#_finalizeSale() - Incorrect fee parameter will result in

Description

The `_finalizeSale()` function uses an incorrect divisor (1000 instead of 10000) when calculating the stable coin amount for liquidity, resulting in 10x more funds being allocated to the liquidity pool than intended.

Vulnerability Details

The vulnerable calculation in _finalizeSale():

```
  function _finalizeSale() internal{

        stopTrade = true;

        IBondingFactory bFactory = IBondingFactory(bondingFactory);

        address feeToken = bFactory.getFeeToken();

        address positionManager = bFactory.getUniswapV3PositionManager();

        address platformAdmin = bFactory.getFeeCollector();



        uint256 stableBalance = IERC20(stableCoin).balanceOf(address(this));

         uint256 amountStableForLiquidity = (stableBalance * bFactory.getLBMPercentage())
/ 1000; // 7.5%
```

Since the given fee is assumed to be set in bps i.e. (0 - 10000) for better precision, then the 7.5 % fee will be `750/1000` which becomes 75% instead of `750/10000`.

Impact

Liquidity pool receives 10x more funds than designed and token creators receive only ~20% of funds instead of ~87.5%

Recommendation

Change the divisor from 1000 to 10000.

Status

Resolved

[H-03] BondingToken#_finalizeSale() - Uninitialized state variable is used during UniswapV3 pool creation.

Description

The `_finalizeSale()` function functions once an appropriate amount of tokens have been sold out.
It swaps all the stablecoins provided by the users into fee tokens followed by creating a UniswapV3 pool between bonding token and fee token.

However, during pool creation it use, fee token amount correctly, but uses `tokenForLiquidity` state variable as `amount1Desired` which is zero since it was never initialised.

Vulnerability Details

The vulnerable part in `_finalizeSale()`:

```
       // Add liquidity to V3 pool

                           INonfungiblePositionManager.MintParams   memory   params   =
INonfungiblePositionManager.MintParams({

            token0: feeToken,

            token1: address(this),

            fee: 3000,

            tickLower: TickMath.MIN_TICK,

            tickUpper: TickMath.MAX_TICK,

            amount0Desired: feeTokenAmount,

            amount1Desired: tokenForLiquidity, //@audit Uninitialized

            amount0Min: 0,

            amount1Min: 0,
```

```
            recipient: address(this),
            deadline: block.timestamp + 15 minutes
        });
```

The `tokenForLiquidity` was declared in `BondingTokenStorage.sol` but it was never initialized anywhere. This means that the pool which is about to created will be a one sided pool.

Impact

The pool is created with only single sided liquidity, which is not intended.

Recommendation

Initialize the state variable to the actual value used.

Status

Resolved

Medium

[M-01]BondingToken#_finalizeSale() - Lack of slippage protection will result in suboptimal swaps.

Description

The `finalizeSale()` function swaps stablecoins into fee tokens without providing slippage parameter, exposing it to MEV attacks.

Vulnerability Details

Here is the vulnerable part of the function

```
       // Swap 7.5% stable to feeToken using V3

                            ISwapRouter.ExactInputSingleParams   memory   swapParams   =
ISwapRouter.ExactInputSingleParams({

            tokenIn: stableCoin,

            tokenOut: feeToken,

            fee: 3000, // 0.3% pool fee

            recipient: address(this),

            deadline: block.timestamp + 15 minutes,

            amountIn: amountStableForLiquidity,

            amountOutMinimum: 0, //@audit no min amount

            sqrtPriceLimitX96: 0

        });
```

Notice how amountOutMinimum is hardcoded to 0. This means that the swap will be successful even when the trade is suboptimal.

Impact

Value loss and reduced liquidity in liquidity provision

Recommendation

Consider using an appropriate slippage parameter.

Status

Resolved

[M-02] BondingToken#buy/sell - Hardcoded lbm cost might be problematic

Description

When buy and sell is happening, the lbm cost is hardcoded instead of fetching the price from the `UniswapV3` pool.

The part of code that is commented out is responsible for fetching price via `UniswapV3` pool, indicating that lbmCost is variable.

Vulnerability Details

The buy and sell functions use hardcoded pricing:

```
   function buy(uint256 _amount, address caller) external WheninTrading{

        require(_amount > 0, "Amount should be greater than 0");



        IBondingFactory bFactory = IBondingFactory(bondingFactory);

        uint256 tokenPrice = bFactory.getTokenPrice(address(this));

        address feeToken = bFactory.getFeeToken();



        // Calculate cost and fees

        uint256 cost = _amount * tokenPrice; // 100e18 * 1e18

        uint256 fee = (cost * bFactory.getFeePercentage()) / 10000;  // 1e36 * 10%



 // uint256 lbmCost = IUniswapPriceFetcher(uniswapPriceFeed).getTokenPriceV3(feeToken, 1 * (10**18), 0);

        uint256 lbmCost = 1 * (10**18);



    function sell(uint256 _amount, address caller) external WheninTrading{

        IBondingFactory bFactory = IBondingFactory(bondingFactory);
```

```
        uint256 tokenPrice = bFactory.getTokenPrice(address(this));
        address feeToken = bFactory.getFeeToken();

        // Calculate proceeds and fees
        uint256 proceeds = _amount * tokenPrice;
        uint256 fee = (proceeds * bFactory.getFeePercentage()) / 10000;

    // uint256 lbmCost = IUniswapPriceFetcher(uniswapPriceFeed).getTokenPriceV3(feeToken,
1 * (10**18), 0);
        uint256 lbmCost = 1 * (10**18);
        uint256 lbmFee =  fee/lbmCost;
```

However, the commented code and an additional contract i.e. UniswapPriceFetcher.sol suggests that the price of lbm is dynamic. This means that hardcoded lbmcost can result in more or less fee taken from the user.

Impact

Fee taken from users is not dynamic according to market conditions. Instead, it is hardcoded to 1e18.

Recommendation

Fetch price from UniswapV3 as the design choice suggested or remove the code if not.

Status

Resolved

QA

[Q-01] BondingToken/BondingFactory- Contract does not follow CEI pattern.

Description

There are certain functions in the contracts that does not follow CEI pattern such as

- buy
- _finalizeSale
- createBondingToken()

Recommendation

Although the external calls are safe, it is recommended to add a nonReentrant modifier.

Status

Resolved

Centralisation

The Libertum project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Libertum project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au